

PROGRAMAÇÃO CLIENTE SERVIDOR · SPRING BOOT

Semana 1

Setup do Projeto

Git · GitHub · Spring Initializr · Estrutura de Pacotes

03 – 09 de maio · Entrega: 09/05



Agenda de Hoje

O que vamos aprender e fazer nesta aula

01

O que é Git e por que usar?

Controle de versão na prática

02

Criando conta e repositório GitHub

Plataforma + boas práticas

03

Configurando o ambiente

Instalação do Git + configurações iniciais

04

Criando o projeto Spring Boot

Spring Initializr + estrutura de pacotes

05

Primeiro commit e push

Conectando projeto ao GitHub



Meta da aula: repositório no GitHub pronto com o projeto Spring Boot rodando

O que é Git?

Controle de versão distribuído

Git é uma ferramenta que registra cada alteração feita no seu código, assim você pode voltar no tempo, colaborar e nunca perder trabalho.



Histórico

Cada versão salva é chamada de commit.
Você vê quem mudou o quê e quando.



Ramos (Branch)

Crie uma cópia paralela do código para
testar sem afetar o original.



Colaboração

Vários devs trabalham no mesmo projeto
sem se atrapalhar.

O que é GitHub?

A nuvem do seu código



GitHub

Plataforma online que armazena repositórios Git na nuvem.
Seu código fica salvo, acessível de qualquer lugar e
compartilhável com a turma e o professor.

Git vs GitHub

Git

Ferramenta instalada no seu computador

GitHub

Site onde você guarda e compartilha o repositório

*Analogia: Git é como o Word com histórico de versões.
GitHub é o Google Drive onde você salva o documento.*

Passo 1 — Instalar o Git

Faça isso antes de tudo



Windows

1. Acesse git-scm.com/download/win
2. Clique em "Download for Windows"
3. Execute o instalador baixado
4. Nas opções, mantenha o padrão e clique "Next" até finalizar
5. Abra o Git Bash (menu iniciar → Git Bash)



Mac / Linux

Mac — via Homebrew:

```
brew install git
```

Linux (Ubuntu/Debian):

```
sudo apt install git
```

Verificar instalação (qualquer SO):

```
git --version
```



Se o comando retornar `git version 2.x.x` — está instalado corretamente!

Passo 2 — Configurar o Git

Apenas uma vez por máquina

Abra o terminal (Git Bash no Windows, Terminal no Mac/Linux) e execute:

```
# Configure seu nome (aparece nos commits)
git config --global user.name "Seu Nome Completo"

# Configure seu e-mail (use o mesmo do GitHub!)
git config --global user.email "seu@email.com"
```

Para verificar que ficou salvo:

```
git config --global --list
```

Por que isso é importante?

Cada commit identifica o autor. Seu nome e e-mail aparecerão no histórico do GitHub — importante para o professor saber quem fez cada parte do projeto.

Passo 3 — Criar Conta no GitHub

github.com → Sign Up

- 1 Acesse github.com e clique em Sign Up
- 2 Informe e-mail, senha e um nome de usuário (ex: joao-silva)
- 3 Confirme o e-mail recebido na sua caixa de entrada
- 4 Pule as perguntas opcionais e acesse o painel principal

✓ **Dica: use seu e-mail institucional ou pessoal. Anote seu nome de usuário — você vai usar em todo o projeto!**

Passo 4 — Criar o Projeto Spring Boot

start.spring.io

Configurações no Spring Initializr

Project

Maven

Language

Java

Spring Boot

3.2.x (estável)

Group

com.seunome

Artifact

nome-do-projeto

Java Version

17

6 Dependências — ADD ✓



Spring Web

Endpoints REST + Tomcat



Spring Data JPA

ORM com Hibernate



H2 Database

BD em memória (dev)



Validation

Bean Validation



Lombok

Reduz código repetitivo



SpringDoc OpenAPI

Swagger UI automático

Clique em **GENERATE** → extraia o .zip → abra na sua IDE (IntelliJ ou VS Code)

Estrutura de Pacotes do Projeto

Cada pasta tem UMA responsabilidade

```
src/main/java/com/seunome/projeto/  
├── controller/  
│   └── (recebe requisições HTTP)  
├── service/  
│   └── (lógica de negócio)  
├── repository/  
│   └── (acesso ao banco de dados)  
├── entity/  
│   └── (tabelas do banco)  
└── exception/  
    └── (erros customizados)
```

controller/

Recebe as requisições HTTP e devolve as respostas. É a porta de entrada da API.

service/

Contém toda a lógica de negócio. O controller chama o service.

repository/

Acessa o banco de dados via JPA. Não há SQL manual aqui.

entity/

Representa as tabelas do banco. São as classes com @Entity.

exception/

Exceções customizadas e o handler global de erros.

Configurando o application.yml

src/main/resources/application.yml

```
server:
  port: 8080

spring:
  datasource:
    url: jdbc:h2:mem:projetodb
    username: sa
    password:
  h2:
    console:
      enabled: true
      path: /h2-console
  jpa:
    hibernate:
      ddl-auto: create-drop
    show-sql: true

springdoc:
  swagger-ui:
    path: /swagger-ui.html
```

Após subir a aplicação, verifique:

<localhost:8080/h2-console>

Console do banco H2

<localhost:8080/swagger-ui.html>

Documentação da API

<localhost:8080/v3/api-docs>

Spec OpenAPI (JSON)

✓ Console: "Started Application in X seconds"

Passo 5 — Criar Repositório no GitHub

github.com → New repository

1 No GitHub, clique em + → New repository

2 Dê um nome ao repositório (ex: "sistema-pedidos")

3 Marque como Public

O professor precisa acessar

4 NÃO marque Add a README (ainda)

O README vem do seu
projeto local

5 Clique em Create repository



Passo 6 — Primeiro Commit e Push

No terminal, dentro da pasta do projeto

```
# 1. Entre na pasta do projeto (onde está o pom.xml)
cd nome-do-projeto

# 2. Inicialize o Git localmente
git init
git branch -M main

# 3. Conecte ao repositório remoto do GitHub
git remote add origin https://github.com/seuusuario/nome-do-projeto.git

# 4. Adicione todos os arquivos e faça o primeiro commit
git add .
git commit -m "feat: initial Spring Boot setup"

# 5. Envie para o GitHub
git push -u origin main
```

git add .

Prepara os arquivos para o commit

git commit -m ...

Salva um snapshot com mensagem

git push

Envia para o GitHub na nuvem

Boas Práticas — Commits

Seu histórico de commits é uma evidência avaliada

✓ Faça assim

```
git commit -m "feat: add Produto entity"
```

```
git commit -m "feat: add ProdutoRepository"
```

```
git commit -m "fix: correct price validation"
```

```
git commit -m "docs: update README"
```

✗ Evite isso

```
git commit -m "."
```

```
git commit -m "atualizei"
```

```
git commit -m "asdfgh"
```

```
git commit -m "final final v2"
```



Prefixos convencionais: feat: (novo recurso) · fix: (correção) · docs: (documentação) · refactor: (refatoração)

O Arquivo .gitignore

Diga ao Git o que NÃO enviar para o GitHub

O Spring Initializr já gera um .gitignore adequado. Verifique se ele contém:

```
# Compilado do Maven
target/

# Configurações da IDE
.idea/
.vscode/
*.iml

# Sistema operacional
.DS_Store
Thumbs.db
```

target/

Código compilado. Gerado automaticamente. Não precisa versionar.

.idea/

Configurações do IntelliJ. São específicas de cada máquina.

.vscode/

Configurações do VS Code. Não devem ir para o repositório.

*.iml

Arquivo de módulo do IntelliJ. Gerado pela IDE.



NUNCA suba a pasta target/ para o GitHub — ela tem centenas de MB e arquivos gerados.

Checklist da Semana 1

Antes de fechar — verifique cada item

- ☐ Git instalado e configurado (nome + e-mail)
- ☐ Conta no GitHub criada
- ☐ Projeto Spring Boot gerado no Spring Initializr
- ☐ 6 dependências adicionadas corretamente
- ☐ Aplicação sobe sem erros ("Started Application...")
- ☐ Estrutura de pacotes criada (entity, service, etc.)
- ☐ Repositório no GitHub — público + .gitignore correto
- ☐ README.md com nome do projeto e integrantes
- ☐ Commit inicial: "feat: initial Spring Boot setup"

5 pts

3 pts

4 pts

Total desta semana: ~12 pontos encaminhados · Entrega até 09/05

Próxima Semana

Model + Repository + CRUD básico

Entidades JPA · Relacionamentos · Repositories · Controllers GET



Git configurado



Repositório criado



App rodando



Checklist ok

Dúvidas? Chame no grupo da turma ou abra uma issue no repositório.